



38. Elastic Net Regression

The Perfect Blend for Optimal Predictions

Previous Section:

 [37. Softmax Regression](#)

Contents:

[1. Elastic Net Regression – The Overview](#)

[2. Elastic Net Regression – The Comprehensive Workflow](#)

[3. Elastic Net with Python](#)

[Summary](#)

[References](#)

Before touching this section, I recommend that you visit the [Lasso and Ridge Regression](#) section. But if you have read it. Then you are more than ready for this one.

As we are about to dive into Elastic Net, let me ask you something. Have you ever felt like choosing between Lasso and Ridge Regression is a bit unfair?

- On one side, Lasso is decisive. It cuts features out completely—clean, simple, efficient.
- On the other side, Ridge is more conservative. It keeps everything, but shrinks it down, controlling complexity without throwing information away.

Two different philosophies. Two different ways of thinking. And now comes the real question: **What if you didn't have to choose?**

Because in real-world data, things are rarely that simple. Sometimes you want feature selection. Remove noise, simplify the model. But at the same time, you also want stability. Handle multicollinearity, avoid overfitting.

Lasso alone might be too aggressive. Ridge alone might be too forgiving.

So instead of asking: "Which one is better?" → We ask a much more interesting question: *"Can we combine both... and get the best of both worlds?"*

And that is exactly where Elastic Net Regression comes in. It's not just another regression technique. It's a balance. A compromise. A hybrid that learns when to be strict and when to be flexible.

Think of it like this:

- Lasso says: *"If it's not important, remove it."*
- Ridge says: *"Everything matters... just not equally."*
- Elastic Net says: *"Let's decide that... together."*

And that subtle shift? That's what makes it powerful.

1. Elastic Net Regression – The Overview

Let's bring everything together. Elastic Net Regression is what happens when we stop thinking in extremes and start thinking in balance.

Up until now, you've seen two very powerful ideas:

- **Lasso (L1)** → aggressive, selective, willing to remove features entirely
- **Ridge (L2)** → cautious, stable, keeps everything but controls magnitude

Both are correct. Both are useful. But both are incomplete on their own. Because real-world data doesn't behave in a way that perfectly fits one philosophy.

The Limitation of Choosing One Side

Let's say you're dealing with highly correlated features.

Lasso will look at them and make a decision:

| *"I'll keep this one... and eliminate the rest."*

Clean? Yes. Stable? Not always. Because if the data changes slightly, Lasso might pick a different feature next time. Same information, different representation, unstable selection.

Now Ridge takes a completely different stance:

| *"Why choose? They are all useful... I'll just shrink them together."*

Stable? Yes. Interpretable? Not really. Because now everything stays in the model, even things you might not care about.

Elastic Net – The Balance Between Selection and Stability

This is where Elastic Net steps in, not as a replacement, but as a reconciliation. Instead of forcing a choice between L1 and L2, it uses both. It says:

“Some features should disappear... but correlated ones should stay together.”

So what happens?

- It **removes irrelevant features** (like Lasso)
- It **keeps correlated features as a group** (like Ridge)
- It **reduces instability** in feature selection
- It **maintains interpretability without sacrificing robustness**

This is why Elastic Net is especially powerful when:

- Predictors are highly correlated
- The number of features is large (even larger than the number of observations, $m > n$)
- You want both **sparsity** and **stability**

The Mathematical Idea

At its core, Elastic Net is just Linear Regression with a smarter penalty. Here is the loss function that drives everything:

$$L(\hat{\beta}) = \frac{1}{2n} \sum_{i=1}^n (y_i - x_i^T \hat{\beta})^2 + \lambda \left((1 - \alpha) \sum_{j=1}^m \hat{\beta}_j^2 + \alpha \sum_{j=1}^m |\hat{\beta}_j| \right)$$

Now don't get lost in the symbols—focus on the idea. There are **two forces** acting on the model:

- The **L2 part** → keeps coefficients small and stable
- The **L1 part** → pushes some coefficients to exactly zero

And controlling the balance between them... is a single parameter:

The Role of α → Think of α as a dial.

- $\alpha = 0 \rightarrow$ Pure Ridge
- $\alpha = 1 \rightarrow$ Pure Lasso
- $0 < \alpha < 1 \rightarrow$ A true hybrid

So instead of committing to one method, you can *tune the behavior of your model*.

Elastic Net succeeds because it solves two problems at the same time: It **reduces variance** (like Ridge). It **introduces sparsity** (like Lasso)

But more importantly, it does something neither method does well alone. It **selects correlated features as groups**, instead of arbitrarily picking one. That single property makes it far more stable in real-world scenarios.

If Lasso is about **simplicity**, and Ridge is about **stability**. Then Elastic Net is about **control**. It gives you the ability to decide: How much to simplify. How much to stabilize. And how to balance both based on the data itself.

And that's why, in high-dimensional problems—like genomics, text processing, or any dataset filled with correlated features—Elastic Net is often not just an option. It becomes the **default choice**.

2. Elastic Net Regression – The Comprehensive Workflow

Now we stop talking about ideas and actually *walk through the process*.

Because understanding Elastic Net is one thing. But seeing how it behaves on real data? That's where it becomes real.

Let's begin with something simple, but structured enough to reveal the pattern:

First-Semester (X)	Second-Semester (Y)	XY
4.5	5.1	22.95
5.2	6.3	32.76
6.0	6.7	40.20
6.4	7.5	48.00
7.1	7.8	55.38
7.3	8.1	59.13
8.0	8.6	68.80
8.4	9.0	75.60

First-Semester (X)	Second-Semester (Y)	XY
9.1	9.4	85.54
9.5	9.8	93.10

(This is the same dataset I use in the Ridge and Lasso Regression section)

From this, we compute the essentials:

- $n = 10$
- $\sum XY = 581.46$
- $\bar{X} = 7.15, \bar{Y} = 7.83$
- $S_{xx} = 24.345, S_{xy} = 21.615$

Step 1 – Start with OLS (The Baseline)

Before anything else, we begin with **Ordinary Least Squares**. Because Elastic Net doesn't replace OLS. It *modifies it*.

- Slope (β_1) Calculation

$$\beta_1 = \frac{n \sum x_i y_i - (\sum x_i)(\sum y_i)}{n \sum x_i^2 - (\sum x_i)^2} = \frac{(10 \times 581.46) - (71.5 \times 78.3)}{(10 \times 535.569) - 5112.25} = 0.8879$$

- Intercept (β_0) Calculation

$$\beta_0 = \frac{\sum y_i - \beta_1 \sum x_i}{n} = \frac{78.3 - (0.8879 \times 71.5)}{10} = 1.4815$$

So the baseline model becomes:

$$\hat{Y} = 1.4815 + 0.8879X$$

Make predictions (This time around, we rounded the results up to four decimals):

X	Predicted $\hat{Y}$ (Predicted)	Actual Y	Error ($Y - \hat{Y}$)	(Error) ²
4.5	5.4771	5.1	-0.3771	0.1422
5.2	6.0986	6.3	0.2014	0.0406
6.0	6.8089	6.7	-0.1089	0.0119
6.4	7.1641	7.5	0.3359	0.1129

X	Predicted $\hat{Y}$ (Predicted)	Actual Y	Error ($Y - \hat{Y}$)	(Error) ²
7.1	7.7876	7.8	0.0124	0.0002
7.3	7.9632	8.1	0.1368	0.0187
8.0	8.5847	8.6	0.0153	0.0002
8.4	8.9399	9.0	0.0601	0.0036
9.1	9.5614	9.4	-0.1614	0.0261
9.5	9.9166	9.8	-0.1166	0.0136

Sum of Squared Errors: $SSE_{OLS} = 0.3698$

This is the **unregularized truth**. The pure relationship before we control complexity.

Step 2 – Introduce Elastic Net

Now we add discipline to the model. Elastic Net doesn't trust raw OLS completely. It asks: *"What if this slope is slightly overfitted?"*

So we introduce the combined penalty:

$$L(\beta) = \frac{1}{2n} \sum_{i=1}^n (y_i - x_i^T \beta)^2 + \lambda \left((1 - \alpha) \sum \beta_j^2 + \alpha \sum |\beta_j| \right)$$

Two forces now exist:

- **L2 (Ridge)** → smooth shrinkage
- **L1 (Lasso)** → selective shrinkage

And the balance is controlled by: $\lambda = 0.5; \alpha = 0.5$. So this is a **true hybrid**.

Step 3 – Solve Using Elastic Net Formula

For a single-feature case, the solution becomes:

$$\beta_{EN} = \frac{S(S_{xy}, \lambda\alpha)}{S_{xx} + \lambda(1 - \alpha)}$$

Where the soft-thresholding function is:

$$S(z, \gamma) = \begin{cases} z - \gamma & z > \gamma \\ 0 & |z| \leq \gamma \\ z + \gamma & z < -\gamma \end{cases}$$

Now here's the intuition (don't skip this):

- First → **L1 shrinks (and possibly eliminates)**
- Then → **L2 stabilizes the result**

Apply the Numbers → Given: $\lambda = 0.5; \alpha = 0.5$

- Solve the Denominator (Ridge Component)

$$S_{xx} + \lambda(1 - \alpha) = S_{xx} + 0.5(1 - 0.5) = S_{xx} + 0.25$$

- Solve the Numerator (Lasso Shrinkage)

$$S(S_{xy}, \lambda\alpha) = S(S_{xy}, 0.25)$$

- Apply the soft-thresholding operator:

$$S(S_{xy}, 0.25) = \begin{cases} S_{xy} - 0.25 & S_{xy} > 0.25 \\ 0 & |S_{xy}| \leq 0.25 \\ S_{xy} + 0.25 & S_{xy} < -0.25 \end{cases}$$

- Define the components and apply the soft-thresholding operator $S(S_{xy}, \lambda_1)$

$$S_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2 = 24.345$$

$$S_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = 21.615$$

Since the OLS slope is positive ($0.8879 > 0$), we use the positive case for the soft-thresholding operator $S(S_{xy}, 0.25)$:

$$\beta_{ElasticNet} = \frac{S(S_{xy}, 0.25)}{S_{xx} + 0.25} = \frac{21.615 - 0.25}{24.345 + 0.25} = \frac{21.365}{24.595} = 0.86867$$

- **Intercept Calculation**

$$\beta_0 = \bar{y} - \beta_{ElasticNet} \bar{x} = 7.83 - (0.86867 \times 7.15) = 1.619$$

- **Final Elastic Net Model:**

$$\hat{Y}_{ElasticNet} = 1.619 + 0.86867X$$

Step 4 – Compare with OLS

Model	Equation
OLS	$Y = 1.4815 + 0.8879X$
Elastic Net	$Y = 1.619 + 0.86867X$

Now observe carefully:

- Slope **decreases**: $0.8879 \rightarrow 0.86867$
- Intercept **increases slightly**

This is not random. This is **shrinkage in action**. Elastic Net pulls the model toward simplicity without destroying the relationship.

Step 5 – Predictions

Now let's see how the model behaves:

X	Predicted $\hat{Y}$	Actual Y	Error ($Y - \hat{Y}$)	(Error) ²
4.5	5.1	5.5280	-0.4280	0.1832
5.2	6.3	6.1361	0.1639	0.0269
6.0	6.7	6.8310	-0.1310	0.0172
6.4	7.5	7.1785	0.3215	0.1034
7.1	7.8	7.7866	0.0134	0.0002
7.3	8.1	7.9603	0.1397	0.0195
8.0	8.6	8.5684	0.0316	0.0010
8.4	9.0	8.9158	0.0842	0.0071
9.1	9.4	9.5239	-0.1239	0.0154
9.5	9.8	9.8714	-0.0714	0.0051

Sum of Squared Errors: $SSE_{ElasticNet} = 0.3788$

At first glance, something might feel wrong: *“Wait... Elastic Net has higher error than OLS?”*

Yes. And that’s exactly the point. Because OLS is obsessed with minimizing training error. Elastic Net is obsessed with **generalization**.

So what happens?

- The line becomes slightly flatter
- The model becomes less sensitive to noise
- The predictions become more stable

Elastic Net is not trying to win on training data. It’s preparing for something much harder: **data it has never seen before**

That’s why it shines when features are correlated, data is high-dimensional, stability matters more than perfection

If OLS is the “perfect student who memorizes everything”. Then Elastic Net is the one who actually understands the lesson.

3. Elastic Net with Python

Elastic Net with Python is straightforward with the Scikit-Learn library:

```
import pandas as pd

sample_data = {'First-Semester Score (X)': [4.5, 5.2, 6.0,
6.4, 7.1, 7.3, 8.0, 8.4, 9.1, 9.5],
               'Second-Semester Score (Y)': [5.1, 6.3, 6.7,
7.5, 7.8, 8.1, 8.6, 9.0, 9.4, 9.8]}

df = pd.DataFrame(sample_data)
```

```
from sklearn.model_selection import train_test_split

# 80% Training, 20% Testing
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
# Features and Targets
X_train = train_df[['First-Semester Score (X)']]
y_train = train_df['Second-Semester Score (Y)']
```

```
X_test = test_df[['First-Semester Score (X)']]
y_test = test_df['Second-Semester Score (Y)']
```

```
from sklearn.linear_model import ElasticNet
from sklearn.metrics import mean_squared_error, r2_score

# Train the Elastic Net Regression model
# l1_ratio: Defines the mix between L1 and L2.
# + l1_ratio = 1: Pure Lasso.
# + l1_ratio = 0: Pure Ridge.
# + 0 < l1_ratio < 1: A hybrid approach that allows for both feature selection
# (setting some coefficients to zero)
# and coefficient shrinkage (grouping correlated variables).
model = ElasticNet(alpha=0.5, l1_ratio=0.5)
model.fit(X_train, y_train)
# Display the Elastic Net Regression equation
A = model.intercept_
B = model.coef_[0]
print(f"Elastic Net Equation: Y = {A} + ({B} * X)")
# Evaluation
y_predict = model.predict(X_test)
# Compute the Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_predict)
print(f"Mean Squared Error on the test set: {mse}")
# Compute the R2 score
r2 = r2_score(y_test, y_predict)
print(f"R2 Score on the test set: {r2}")
```

Elastic Net Equation: $Y = 2.647545357524012 + (0.724119530416222 * X)$

Mean Squared Error on the test set: 0.019659869824560947

R² Score on the test set: 0.9918169116234918

```
# Plot the OLS line and the Elastic Net line
import numpy as np
from sklearn.linear_model import LinearRegression
```

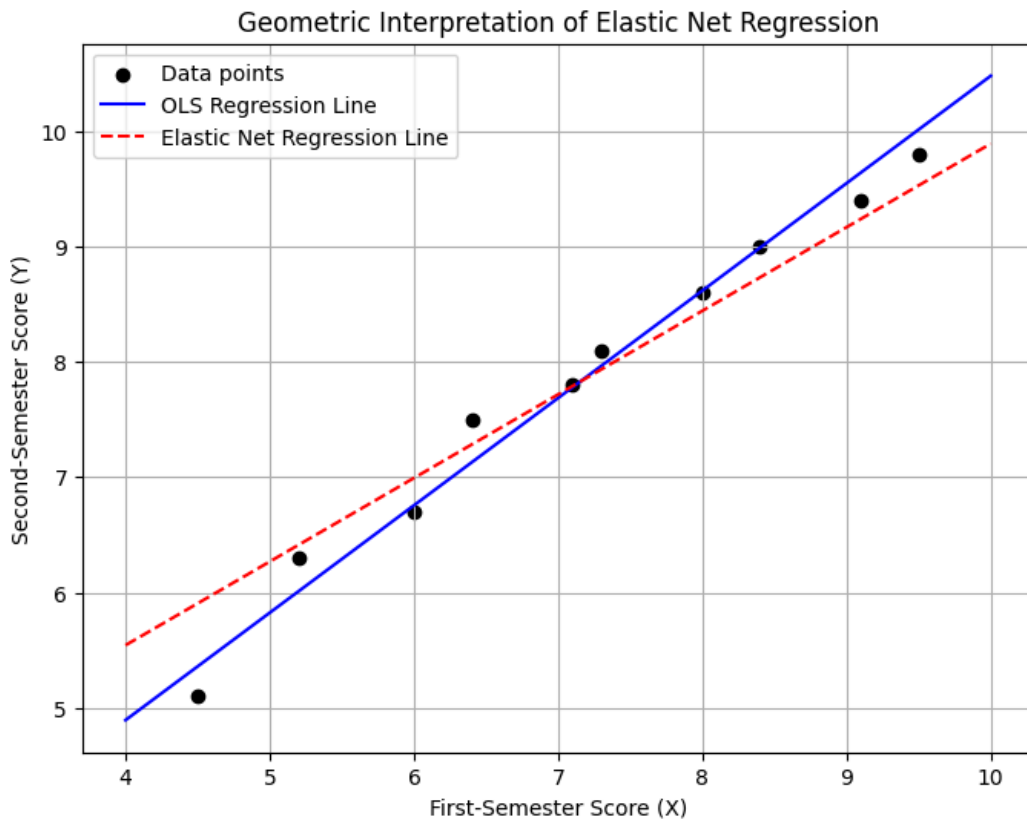
```

import matplotlib.pyplot as plt

# Train the Linear Regression for plotting
ols_model = LinearRegression()
ols_model.fit(X_train, y_train)
x_line = np.linspace(min(df['First-Semester Score (X)'])-0.5,
                      max(df['First-Semester Score (X)']+0.5, 100))
y_ols = ols_model.intercept_ + ols_model.coef_[0] * x_line
y_en = model.intercept_ + model.coef_[0] * x_line

# Plot
plt.figure(figsize=(8,6))
plt.scatter(df['First-Semester Score (X)'], df['Second-Semester Score (Y)'],
            color="black", label="Data points")
plt.plot(x_line, y_ols, color="blue", label="OLS Regression Line")
plt.plot(x_line, y_en, color="red", linestyle="--", label="Elastic Net Regression Line")
plt.xlabel("First-Semester Score (X)")
plt.ylabel("Second-Semester Score (Y)")
plt.title("Geometric Interpretation of Elastic Net Regression")
plt.legend()
plt.grid(True)
plt.show()

```



Summary

Elastic Net Regression feels far less daunting, once you've truly understood Lasso and Ridge. Because in the end, it's not introducing a completely new idea—it's simply *bringing two ideas together*.

Lasso teaches you how models can become simpler by removing unnecessary features. Ridge teaches you how models can remain stable by controlling coefficient magnitude. Elastic Net doesn't replace either of them. It *listens to both*. It takes the sparsity of Lasso... the stability of Ridge... and blends them into a single, more flexible approach.

So instead of asking: "*Should I remove features... or keep them?*" → Elastic Net answers: "*Let the data decide... but guide it carefully.*"

And that's why it becomes so powerful in real-world scenarios—especially when things get messy:

- When predictors are highly correlated
- When datasets become high-dimensional
- When stability matters just as much as interpretability

At that point, Elastic Net is no longer just an alternative, it becomes the **natural choice**. Because sometimes, the best solution isn't picking a side—It's knowing how to balance both.

References

<https://www.geeksforgeeks.org/machine-learning/implementation-of-elastic-net-regression-from-scratch/>

<https://codesignal.com/learn/courses/diving-deep-into-regression/lessons/elastic-net-regression>

<https://www.youtube.com/watch?v=1dKRdX9bflo>

<https://www.youtube.com/watch?v=ctmNq7FgbvI>

Next Section:

 [39. Sequential Data Modeling](#)